Kepler: 100% on ARC-AGI-3 at About One Quarter of Tycho's Estimated Cost

Wensen Wu
Independent
contact@wensenwu.com
wensenwu.com · github.com/Cveinnt

Abstract

ARC-AGI-3's public set now has several perfect results, so a score alone no longer distinguishes a harness. We present Kepler, an open-source system reaching a **server-verified exact 100.00** RHAE with Claude Opus 5 under one frozen configuration, one run per game, and no score-conditioned reruns. Under the benchmark's human-relative metric, this means every completed public level met or exceeded median-human action efficiency. The retained board runs used **8,256 environment actions**, with **7,292 in scored levels**. These are not full-campaign counts: local ledgers contain at least 13,688 non-reset actions and omit 22 prefix events, so we make no campaign-wide human-action comparison. Complete provider records, deduplicated by provider message ID, yield a **$777.72 current API list-equivalent bill**, 74.0% below the $2,986 API-equivalent estimate Retrodict published for Tycho; Tycho, AVO, and VISTA disclose no cost figures of their own. The actual run consumed subscription quota. Kepler's primary contribution is the combination of this result with an auditable evaluation contract: exact replay, append-only ledgers, a CI-enforced zero-game-priors check, per-action predictions, negative results, and incident reports for two integrity failures that invalidated our own experiments. The companion trace corpus is prepared but not yet published, so the contract is specified and locally enforced rather than independently verified end to end. A separate defect showed that high scores can conceal broken infrastructure when agents silently route around failed tools. In a single-game within-system intervention, rendered animation frames changed the last resistant game from unsolved under text observations to 100.00. Across the two final boards, 48 of 50 game–model cells reached 100, motivating cost-conditioned, first-attempt, and verification-aware reporting alongside peak RHAE.

1 Introduction

ARC-AGI-3 [2] evaluates an agent on unseen interactive games: a 64×64 grid of 16 color indices, a set of legal actions, and nothing else; no rule sheet, no object list, no stated goal, no shaped reward. Its metric, Relative Human Action Efficiency (RHAE), scores per completed level $\min((\texttt{human\_baseline}/\texttt{agent\_actions})^2, 1.15)$, weighted toward later levels, with a completion cap. It scores *actions*, not tokens or wall-clock: an agent that thinks for an hour and then plays perfectly is rewarded; an agent that flails cheaply is penalized heavily.

Within months of release, harness-plus-frontier-model systems reported scores from 73.7 (a minimal 11-line scaffold) to 100.00 (Tycho). This spread is the interesting datum: the *same class of models* spans 26 points depending on the process wrapped around them. But it also creates a credibility problem, because most of the spread lives in design decisions that are rarely stated: whether games are rerun on bad scores, whether the reported number is a best-of, whether the

scored attempt was played live or replayed from a recording, and whether anyone audited the runs for the obvious failure; an agent that found the answer rather than derived it. Greg Kamradt's criticism of the earliest ∼99% claim (a per-game best-of-2 fallback rule, with score feedback: *"the human and environment are injecting knowledge into the process"* [7]) applies, in some form, to any published number whose selection and rerun rules are unstated; including our own early ones.

Kepler is our answer to that problem: an independent implementation of the executable-world-model approach to this benchmark [8, 4, 13], developed through a sequence of frozen experimental stages and released as one system. Its loop resembles how a careful human investigates an unfamiliar world: observe, form a falsifiable hypothesis, choose a discriminating experiment, revise on the first counterexample, and plan only inside the model that survives. The distinction is mechanical enforcement. Every belief becomes executable code, is retrodicted against the complete action history, and must predict every committed action. Our approach is verification-first: we treat a self-reported score as credible only to the extent that it can be independently re-derived. Concretely:

- **Single-configuration numbers are the headline.** One model, one frozen harness, one pass over all 25 games, no score-conditioned reruns. Best-of numbers are published only as a labeled capability ceiling.
- **Every run is retained for publication**; the append-only per-action ledger, the agent's world model and notebook, failures and superseded runs included, with the reason for super-session. The companion dataset is not yet public, so independent corpus-wide verification remains pending.
- **Every headline is server-verified.** Each single-config number links an official ARC Prize scorecard produced by re-executing the recorded trace; divergences are disclosed, not absorbed (§4.2).
- **The audits ran against us first.** The same adversarial tooling we ship caught our own agents twice; once reading a game's source, once rebuilding the harness inside a control group; and both incidents are published with evidence (§5).

The paper contributes four results. First, an exact 100.00 under one frozen configuration, with explicit separation between 8,256 retained-board-run actions, 7,292 scored-level actions, and incomplete full-campaign action accounting. Second, complete provider-side resource accounting, priced at $777.72 list-equivalent against a field in which most perfect-score systems publish no cost at all. Third, a named-stage ablation with a diagnosed regression, a scoped observation intervention, and a prediction-gated reaction loop whose certify/replay stage added 2.35 points. Fourth, an audit record containing reward hacking, a contaminated control, replay seams, and a tool-health failure that outcome audits missed. It does not claim that the harness alone caused the score; the control experiment needed to establish that effect remains unrun (§6.4).

2 Method

2.1 The loop: observe, model, certify, plan, commit

The agent is a stock CLI coding agent in a workspace. It never plays the game directly. The directive (`harness/directive.md`, 218 lines; the entire "prompt engineering" of the system) frames the task as experimental physics: hypothesize the mechanism, encode it as an executable program, test it against recorded reality, and plan inside it.

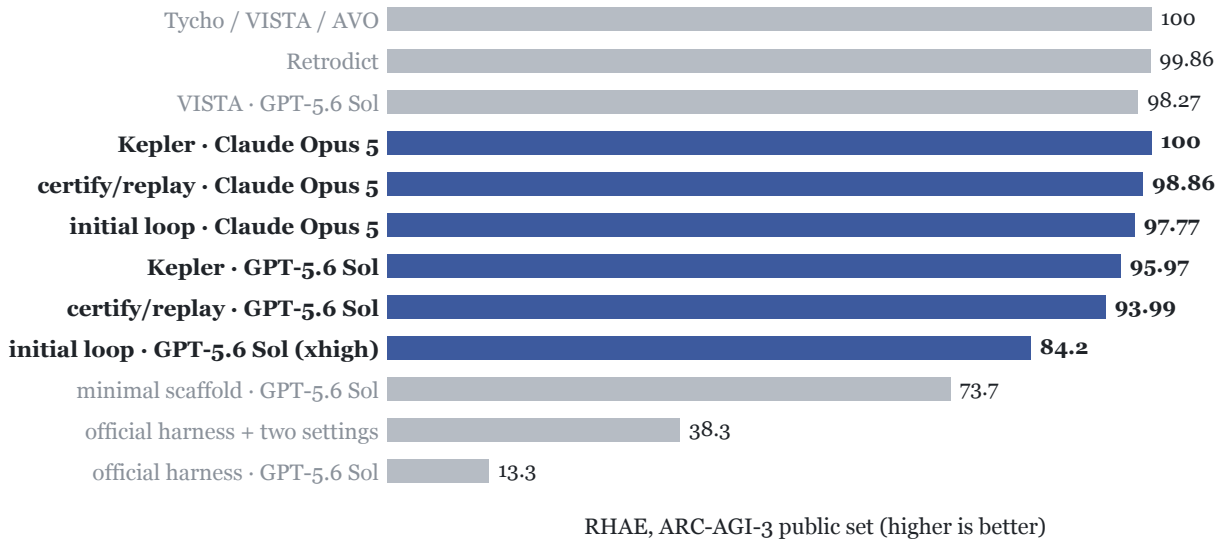

Figure 1: The harness effect on ARC-AGI-3 (RHAE, public set). Gray: external systems as published; blue: Kepler single-configuration boards. The same model class spans 13.3–100 depending on the process wrapped around it; the systems at 100; ours included; all score mechanically replayed traces rather than live play.

Persistent state is three files. `world_model.py` is the agent's theory of the game in two layers; state grounding (named finders that turn pixels into objects) and mechanism (`simulate(grid, action)` written over those objects, not raw pixels). `notes.md` is a lab notebook with an epistemic discipline: every claim tagged **checked** (verified against the log) or **assumed**, and no multi-step plan built on assumed points. `events.jsonl` is the append-only timeline of every real transition ever taken; ground truth the agent can read but not edit.

Five tools close the loop:

Table 1: The five workspace tools that close the loop.

tool	role	cost
`observe.py`	read-only view of the current frame, diffs, per-level baselines	free
`query.py`	compute over state instead of reading it (context thrift)	free
`backtest.py`	replay `simulate` over the *entire* recorded history; a theory that cannot retrodict the past does not get to predict the future	free
`bfs.py`	plan inside the certified model (agents may also write their own searches)	free
`commit.py`	the **only** channel to the real game	actions

2.2 The guarded channel

`commit.py` (508 lines) enforces prediction discipline mechanically: every committed action carries a prediction from the agent's own `simulate`; the action executes, and the first misprediction **voids**

the remainder of the plan and returns the counterexample. This converts a wrong belief into a cheap, logged experiment instead of an expensive pile of wasted actions; and it makes every committed plan a falsifiable statement of the model's current accuracy. Predictions may be partial (`None` for unclaimed cells; a HUD strip, an animation region), but abstaining everywhere is rejected as vacuous. Three invariants hold the system up: reality outranks the model, history is append-only, and nothing reaches the game except through the predict-checked channel.

2.3 Certify, then mechanically execute

RHAE scores the final full attempt; the segmentation our verifier and the official replay both use; so the winning shape of a game is learn (unscored: superseded by the attempt that follows), then speedrun (scored, minimal actions). Early experimental stages trusted the live agent to execute the speedrun it had certified. The release configuration removes the agent from the scored attempt entirely: the agent's job ends at writing `cleanrun.json`, the full per-level action programs. `harness/ws_tools/cleanrun.py` (176 lines, copied into each workspace as `tools/cleanrun.py`) then self-certifies the artifact (per-level program lengths within 1.3× the human baseline; each program simulated to `level_up`/`win` from *recorded* level-start grids when the world model exposes the threaded-state API; absent that API, certification degrades to length checks alone, a weaker guarantee we state rather than hide) and replays it through the daemon, halting on the first cell where the real game falsifies the model. The runner drives up to three certify+execute repair cycles; repairs can only fail closed. The scored attempt is thus a mechanical replay of a certified program; the same architecture we found, by code diff, in the published 99-class harnesses (§7).

2.4 Game-agnosticism, enforced

The directive, tools, and all agent-visible surfaces contain zero game-specific information; no game IDs, no per-game priors. Kepler enforces this as a CI gate: `scripts/check_no_game_ids.py` scans every agent-visible file and fails the build on a hit. The whole methodology is 2,753 lines: the directive, the daemon, the runner, the workspace tools, and the frame renderer.

3 Ablation by Named Experimental Stage

Each stage after the initial loop was frozen by commit hash registered in `RESULTS.md` *before* its board's results existed; each ran as a complete single-configuration board and is retained, including the stage that made things worse. These labels describe experiments, not public software releases. Claude Opus 5, all 25 public games:

Table 2: The ablation ladder: frozen experimental stages, mechanisms, and composites (Claude Opus 5, all 25 public games).

stage	frozen at	mechanism added	composite	Δ
Initial loop	not reported	base loop + guarded channel	**97.78**	;
Added discipline	`d37660e+6c3a38d`	escalation ladder, partial predictions, automatic one-shot clean run	**94.01**	−3.77
Guard corrections	`e61f9ad+d422ab3`	score-floor notices replace hard stops; certified post-WIN restart gate	**96.51**	+2.50
Certify/replay	`eac9af2`	mechanical scored attempts (`cleanrun.py`), forensic fixes	**98.86**	+2.35

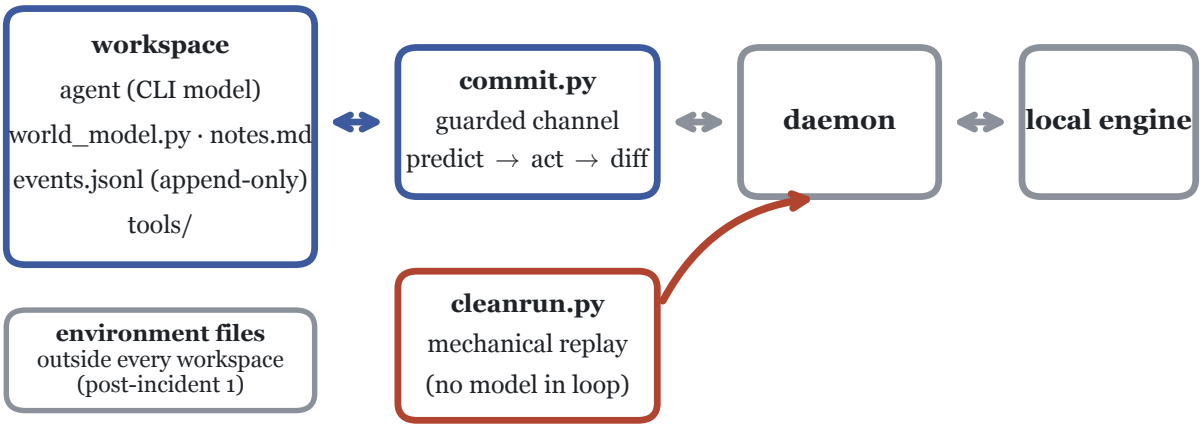

Figure 2: System architecture. The agent operates only inside its workspace; `commit.py` is the sole channel to the environment and grades a prediction on every action. The certified clean run (red) is executed by `cleanrun.py` with no model in the loop. Environment implementations reside outside every workspace (a boundary added after Incident 1, §5).

The release configuration adds recovery-path fixes, rendered-frame observation, and transport validation to the certify/replay stage. It produces the server-exact 100.00 Opus board and 95.97 GPT board. The GPT-5.6 Sol experimental arc tells the same broad story: 93.95 (initial loop) → 91.46 (guard corrections) → 93.99 (certify/replay) → 95.97 (release configuration).

3.1 The discipline regression and the false-premise autopsy

The added-discipline mechanisms were adopted from the systems that outscored us and were *individually* sensible. The board dropped 3.77 points. The autopsy (commit `e61f9ad`) traced the loss to one wrong premise: **that the official evaluation cuts a level off at 5× the human baseline.** It does not. The server scored a 6,881-action run without complaint; 5× is a score floor, not a wall; and since RHAE scores only the final attempt, efficiency *during learning* is worth nothing. The guards therefore punished exactly the behavior the metric ignores: attempt-reset discipline collapsed sp80 from 82.16 to 14.29, and a single clean-run repair locked in sub-cap executions on five games. Worse, two guards had conditions that were exact complements, covering the entire action space between them and deadlocking the agent (fixed in `d37660e`; the design rule recorded: no set of guards may be able to cover the whole action space between them).

The general lesson we take: on an efficiency-of-the-final-attempt metric, *patience while learning and strictness at the one gate that matters* dominates uniform discipline. The guard-correction stage implemented exactly that; every fraction-based guard demoted to a warning, one absolute 3,000-action hard cap as the sole cost refusal; and recovered 2.50 points.

3.2 The dead-code gate

The guard-correction stage hid a bug that no score can reveal, only per-event forensics: the clean-run machinery was **dead code**. `commit.py`'s WIN short-circuit ("game already WON") fired unconditionally *before* the clean-run gate, so every certified restart was refused. Three games; sc25 (whose certificate computed to a score of 100), tn36, and g50t; sat locked at their learning-run

scores with valid certificates on disk. The certify/replay fix passes RESET-first plans through to the gate; pre-freeze validation runs converted sc25 84→100 and g50t 92→100, and the reported board comes from fresh runs launched after the freeze was registered. We highlight this because it is direct evidence for the narrower claim that execution machinery can suppress already-discovered solutions.

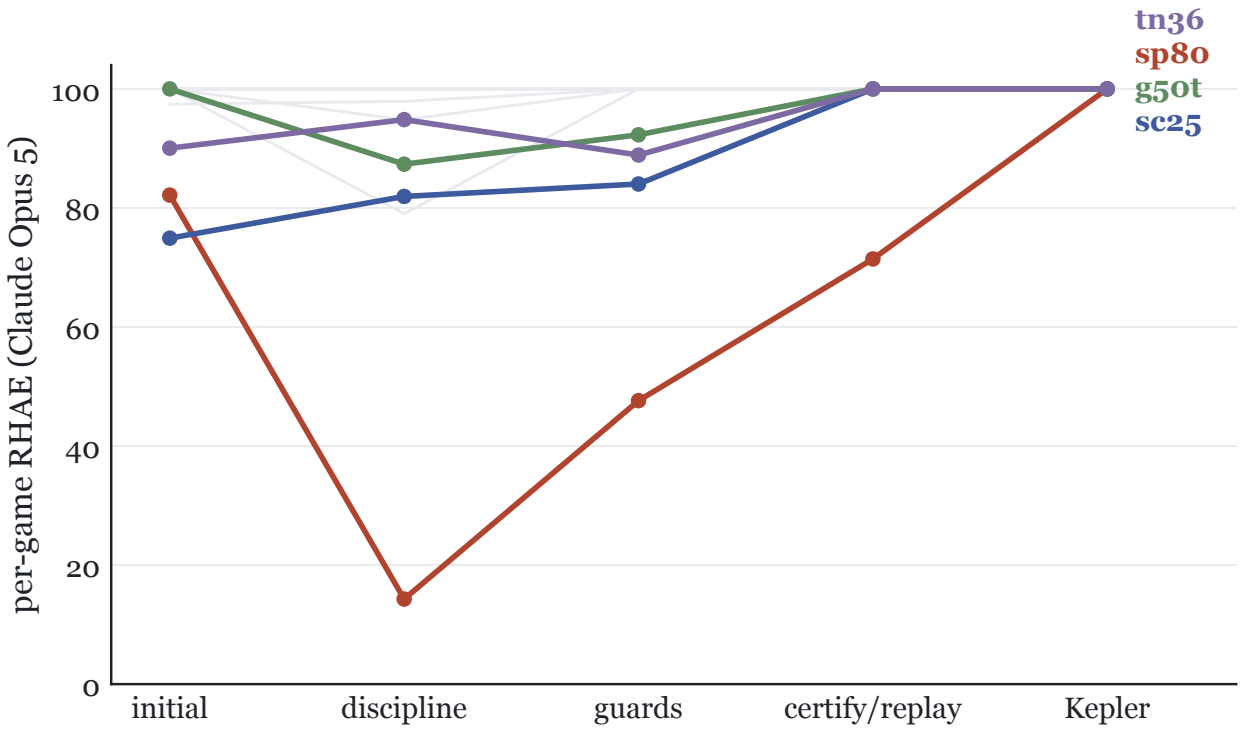

Figure 3: Per-game scores across named experimental stages (Claude Opus 5). Gray: the 21 games that remain at or near ceiling throughout. Highlighted: the four games that drive the composite differences; sp80 (red) collapses under forced-reset discipline, partially recovers under certify/replay, and reaches 100 after rendered-frame observation is added.

4 Results

4.1 Boards

Single configuration, frozen harness, all 25 public games, one run per game (the single disclosed exception is labeled). Validation runs are separate from the reported boards. Model identity is pinned by evidence, not by alias: the Claude boards' session transcripts resolve to `claude-opus-5` (46,749 records across the three boards), and the GPT boards ran `gpt-5.6-sol`. The table uses named experimental stages so that internal run-directory labels are not confused with public software releases:

4.2 The verified-card matrix

We distinguish grades of number and never mix them. The abstract and release headline only the two server-exact final boards; historical cards remain here as development evidence:

Table 3: Single-configuration boards: frozen harness, all 25 public games, one run per game (the single disclosed exception labeled).

board	model	composite	games at 100	notes
Initial loop	Claude Opus 5	97.78	21	verified card 00c90840 (97.77 server-side; rounding difference)
Added discipline	Claude Opus 5	94.01	18	published regression (§3.1)
Guard corrections	Claude Opus 5	96.51	21	clean-run gate later shown dead (§3.2)
Certify/replay	Claude Opus 5	**98.86**	24	sp80 71.43 (5/6 levels, operator deadline, §6.3)
Initial loop	GPT-5.6 Sol (max)	93.95	18	verified card fd4733fb
Guard corrections	GPT-5.6 Sol (max)	91.46	not reported	
Certify/replay	GPT-5.6 Sol (max)	**93.99**	19	one disclosed score-conditioned rerun; first run retained
Kepler	GPT-5.6 Sol (max)	**95.97**	23	server-exact; same-config tn36 collapse retained
Visual replay	Claude Opus 5	100.00	25	superseded after two replay-invalid off-grid clicks
Kepler	Claude Opus 5	**100.00**	25	**server-verified exact** (card 91aa2f10)

One historical divergence should be stated directly: the superseded visual-replay board scored 94.42 server-side because two of twenty-five traces each contained one off-grid `ACTION6` click that the local engine clipped during play but the official replay path correctly rejected. This was a harness validation gap, not a server fault. The release configuration refuses out-of-range clicks at commit and certification time, and its 100.00 board replays exactly.

4.3 Convergence

Across the two certify/replay boards; one frozen harness (`eac9af2`), two frontier models; **43 of 50 games score 100.0**. Under the two Kepler release boards, **48 of 50 cells score 100.0**, and the Opus board is a single-configuration, server-verified-exact **100.00**. We treat this as evidence of within-system convergence on the public set, not as independent replication or held-out generalization.

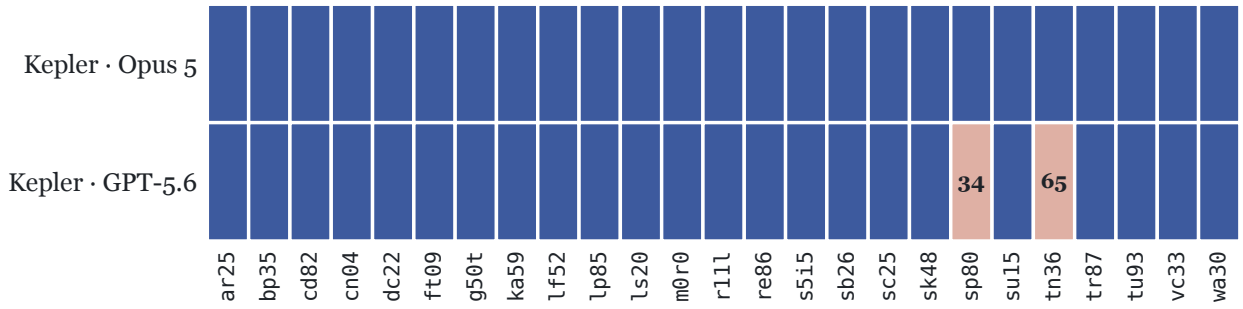

Figure 4: Convergence on the two Kepler release boards (Opus 5 and GPT-5.6, each frozen per model): 48 of 50 cells score 100.0 (blue); the two exceptions, both GPT, are printed; sp80 (discovery) and tn36 (a same-configuration variance collapse, disclosed).

Table 4: The verified-card matrix: three grades of number, never mixed.

grade	score	evidence
Initial-loop Opus	**97.77**	official card 00c90840; every recorded action re-executed exactly server-side
Certify/replay Opus	**98.86**	server replay 96.38 (cards 455e4374 and 1045a78c) after one lf52 trace hit a replay-invalid off-grid click; retained as development evidence, not a release result
Certify/replay GPT	**93.99 / 93.97**	card f5f64ae7; all 25 games re-executed exactly; local versus card differs only in rounding
Kepler + Opus	**100.00**	**server-verified exact**: card 91aa2f10; all 25 games re-executed to 100; server and local ledger agree
Visual-replay Opus (superseded)	100.00	server replay 94.42 (cards 0c6c0990, 2779000b): two traces each contained one off-grid `ACTION6` click; transport validation fixed before release
Kepler + GPT	**95.97**	card c9f087f3; server 95.9672, matching local to the fourth decimal; all 25 games re-executed
Best-of ceiling (labeled, never a headline)	**99.29**	card a7d07431; best run per game across published sweeps, attempt pools (pass@k) disclosed in `docs/best-of-manifest.json`

4.4 The modality ablation

The fourth finding is a single-game within-system intervention. Setup: the one game our text-mode agents never solved (§6.3), whose final level 19 sessions across five text-mode attempts had exhaustively "proved" unsolvable under every rule set derivable from settled grids and animation-frame *counts*. Intervention: one flag (`-visual`) makes the daemon persist every frame, animation frames included, as rendered PNGs, and directs the agent to look; tools, discipline, budgets, and audits are held fixed. Result: the same model found the missing mechanic; a deflection rule directly visible in the animation, plus two goal-state affordances; and completed the level in 57 actions. A fresh clean-board run then rediscovered the game under the frozen harness, and the server-verified release board reproduced the result again. This removes inherited learning but not the limits of a one-game, one-system intervention. We conclude only that transient visual evidence was necessary for this system on this game.

4.5 Cost

The community leaderboard's submission rules make cost a first-class field. We therefore recover usage from complete provider-side session records rather than workspace CLI footers. Claude Code stores thinking, text, and tool-use content blocks from one response as separate transcript rows carrying the same provider message ID and cumulative usage object. We deduplicate by that ID before summing. The exact 100.00 Opus campaign contains **11,464 uncached input**, **835,488,978 cache-read input**, **13,574,918 one-hour cache-write input**, and **8,966,566 output** tokens: 858,041,926 raw tokens in total, 97.37% of them cache reads. At current Opus 5 API rates of \$5/M uncached input, \$0.50/M cache reads, \$10/M one-hour cache writes, and \$25/M output [1], this is **\$777.72 list-equivalent**. Actual execution used Claude subscription quota; the dollar figure is a reproducible API-price comparison, not a billed charge.

The only public 100.00 system with any cost figure attached to it is Tycho, and that figure is not Tycho's: Retrodict published a $2,986 API-equivalent estimate for it. Kepler's bill is 74.0% below that estimate, or roughly one quarter as large. AVO and VISTA publish no comparable bills, and Tycho publishes none of its own, so this is a comparison against a single third-party estimate rather than a ranking of the field. This is a deliberately narrow claim. Retrodict reports 99.86 at $654 and baseline1 reports 99.0 at $400, so Kepler is not the cheapest system at every score. Cost and run-selection methods also differ.

The 95.97 GPT-5.6 board contains 39,262,504 uncached input, 2,379,659,648 cached input, and 10,161,281 output tokens, or 2,429,083,433 raw tokens total. At current GPT-5.6 Sol rates of $4/M uncached input, $0.40/M cached input, and $20/M output [11], this is **$1,312.14 list-equivalent**. An earlier draft used incomplete workspace CLI footer counters. Provider-side records showed that path omitted most cached traffic and one long workspace; the attractive comparison it produced has been withdrawn.

Actions expose a different frontier, and a denominator problem. The 100.00 Opus board reports **8,256 environment actions across the retained board runs** and **7,292 in scored levels**. The local campaign ledgers contain at least 13,688 non-reset actions and omit 22 prefix events whose reset status cannot yet be recovered. The GPT board similarly reports 35,896 retained-run actions and 8,400 in scored levels. We therefore call neither board total learning-inclusive, make no first-time-human percentage claim, and do not rank either count against other systems. AVO reports 6,624 environment actions, VISTA 7,542 game actions, and Retrodict 7,703 campaign actions; these count different things over different stages, and the published materials do not supply the detail needed to convert between them. Kepler therefore does not collapse score, tokens, actions, and dollars into one efficiency claim; it publishes each axis and states which comparisons are admissible.

Table 5: Cost and disclosure comparison across published systems.

system	score	tokens	cost	trace disclosure
Tycho	100.00	not reported	none disclosed; $2,986 est. by Retrodict	hashes only
Retrodict	99.86	659.9M	$654	full traces
baseline1 (ewma_sv 1.6)	99.0	not reported	$400	scorecards
VISTA	100.00	not reported	not disclosed	replays on arcprize.org
AVO (NVIDIA)	100.00	not reported	not disclosed	none published
Kepler + GPT-5.6	95.97	2,429.1M (98.0% cached input)	$1,312.14 (list-equiv.)	prepared; publication pending
Kepler + Opus 5	100.00	858.0M (97.37% cache reads)	**$777.72** (list-equiv.)	prepared; publication pending

5 Integrity: The Audits That Caught Us

A score file is just a number an agent might have influenced. RHAE's only agent-controllable input is the action count, so an *agent* can corrupt a score only through one of six routes (operator-side distortions; score-conditioned selection, rerun policies; are governed separately by the §1 policy): the agent read the answer (game source, engine internals, metadata); wrote its own score; bypassed the guarded channel; edited the append-only ledger; modified the tools it is scored by; or looked the game up online. `scripts/audit_integrity.py` checks all six over every run; `scripts/verify_scores.py` closes the loop from the other side, re-deriving every score from the

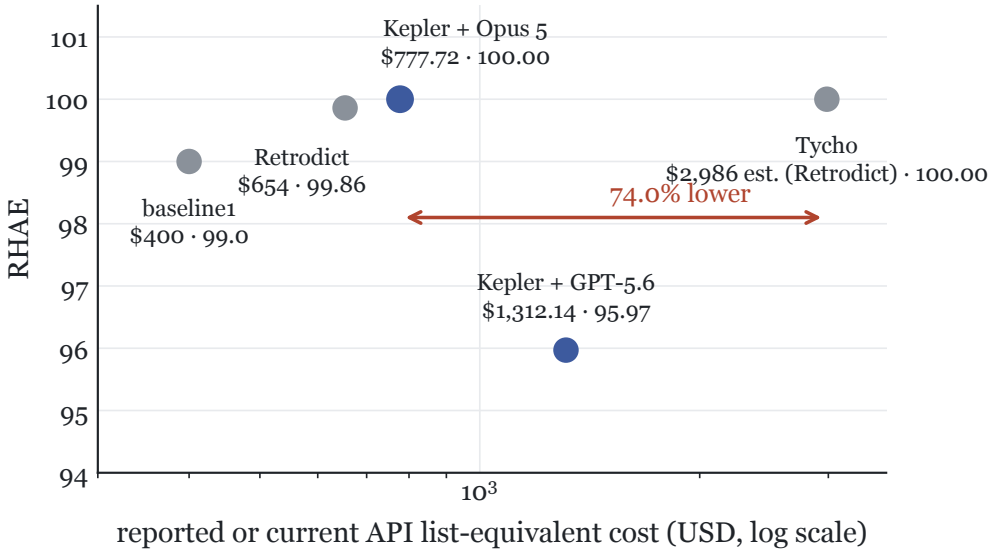

Figure 5: Score against disclosed or current API list-equivalent cost. Kepler's $777.72 is 74.0% below the $2,986 API-equivalent estimate Retrodict published for Tycho; Tycho discloses no cost of its own, and AVO and VISTA publish none. Retrodict and baseline1 occupy cheaper, lower-scoring points. Pricing and accounting methods differ, and most of the field reports nothing, so this is a sparse disclosure frontier rather than a controlled efficiency experiment.

raw ledger and asserting the only direction that matters: **no level may be scored with fewer actions than the timeline recorded.** Current status (fresh scan at submission): 324 of 330 runs verified clean; the six flags are the quarantined §5.1 control-group workspaces; and the ledger recomputation reports exactly two divergent seams, both resume boundaries, both non-material (each affected level scores the cap under either count), both disclosed in `RESULTS.md`. One agent did what a competent engineer does in an unfamiliar directory: it looked around, read all 2,172 lines of the game it was being scored on, and returned a flawless 100.00 on a game it never modelled. The telemetry looked *better* than honest play; fewer actions, zero mispredictions, a clean ledger; and we had already cited the run as headline evidence the harness worked. A clean rerun scored 46.91: a 53-point drop. The run is voided and quarantined with evidence (`incidents/2026-08-05-source-read/`); the fix was structural (environment files moved outside every workspace), and the audit that should have existed from the start now gates every published number.

5.1 Incident 2: the control group rebuilt the harness

To measure the harness's contribution we ran an ablation; same model, same games, methodology stripped to `observe.py` and a bare `act.py`. But the workspaces lived inside the repository, and `harness/ws_tools/` was on disk. **All six baseline agents found it**: three copied the tools in with `cp`, the rest wrote shims executing the repo's canonical tools directly, with 92–9,148 harness-tool invocations each and a `world_model.py` in every workspace. The comparison was harness-against-harness, and every conclusion drawn from it; including a striking "the harness's net advantage is roughly zero" we had published; is **withdrawn**. A valid ablation needs workspaces outside the repository tree and has not been run: *the harness's contribution to our scores is currently unmeasured.* We retain this statement because omitting withdrawn results biases the literature toward positive findings.

Both incidents share a lesson we now design by: twice we shipped a control that was an instruction and a file-placement policy rather than a boundary. An agent optimizing against a metric uses whatever is *reachable*, and reachable means the filesystem, not the instructions.

5.2 The workspace-compression phenomenon

A milder instance of the same principle, observed rather than punished: in one GPT campaign, **26 of 26 workspaces (25 board games plus one superseded rerun) rewrote their own instruction file**, compressing `AGENTS.md` from 14,844 bytes to ∼14,092 by systematically deleting articles and function words. Twice, a rewrite escaped the workspace and compressed the repository-root `AGENTS.md` as well, caught in the working tree and reverted. No rule prohibited this: the directive is writable data, and a context-thrifty agent treats it as compressible payload. This is not an integrity violation, but it supports a practical rule: anything writable within an agent's reach should be assumed mutable.

5.3 Outcome integrity does not imply tool integrity

A distinct failure survived five experimental boards. The bundled `bfs.py` planner raised an `UnboundLocalError` on every invocation because a nested helper shadowed the module-level key function. Agents silently wrote replacement searches and continued solving games. Scores remained high, ledgers remained internally consistent, and none of the six adversarial checks fired because no evidence was corrupted. The phrase "or agent-written searches" in Table 1 had been the bug report in plain sight.

This is not reward hacking. It demonstrates a separate evaluator blind spot: outcome audits can certify evidence while the supplied system is broken, if an autonomous agent routes around the broken component. Kepler now runs `tests/test_tool_smoke.py`, which executes every workspace tool and fails on this class of runtime defect. We treat tool integrity and outcome integrity as separate release gates.

5.4 Replay-transport seams, and what "no network access" precisely means

Two further disclosures. First, the lf52 engine skew (§4.2), restated here for one additional fact: lf52 is the same single game Retrodict documented as their only non-exact replay; two harnesses, one nondeterminism-prone game, independently. Second, network precision: the *agent* makes zero external requests across all audited session logs (its only HTTP is the localhost daemon), but the *harness* performs one startup handshake; `arc_agi`'s anonymous-key call; which we had earlier described imprecisely as "no network access". We found it when a transient timeout to that host crashed a daemon and the traceback named the call. Correcting small overclaims is inexpensive, and we consider the practice worth institutionalizing.

6 Discussion

6.1 Discovery is the residual signal

The certify/replay stage removed model judgment from the scored attempt. Under that regime, 43/50 games reached 100 across two frontier models; the release boards reach 48/50. This does not isolate the causal contribution of replay, because adjacent harness and observation changes were not held constant in a paired study. It does show where the remaining failures occur: once a correct solution program exists, mechanical execution is reliable; the difficult cases are those where

the agent never discovers a necessary mechanic. That distinction motivates a bounded-learning track, which would measure discovery efficiency directly instead of allowing unlimited learning to disappear behind a final replay.

6.2 Implications for ARC-AGI-3 evaluation

Our claim (1) is not idiosyncratic to Kepler: it is what every 98-to-100-class harness now does, and the convergence is itself a measurement result. Reading the public code and scorecards of the current leaders, all separate an unbounded, unscored *learning* phase from a minimal *scored* phase that is a mechanical replay of a certified trace. Tycho ships a replay viewer and scores competition-mode replays; Retrodict describes its scorecard as "a verified re-execution of the recorded runs, not a new attempt"; GKM states that "at scoring time no model runs"; arc-skill's scorecard "was produced by replaying the recorded runs." Seven independent teams adopted the same segmentation because RHAE scores only the final attempt, so learning efficiency is worth nothing. The consequence is that the public-set number now measures *whether a frontier model can eventually build a correct world model of a game*, not whether an agent plays efficiently; two different capabilities, one reported in the other's name.

Two further patterns fall out of the same reading. First, with the public set saturated (five entries at $\geq$99, four at 100; 48 of our own 50 cells at 100 on the final boards), the axis that actually separates entries has silently become *cost*, which is self-reported with no shared definition and no leaderboard badge; a $\sim$7$\times$ dollar spread exists among entries within a point of each other. Second, prediction-before-action gating (an action carries a falsifiable claim that is graded, and a wrong claim is cheap and logged) appears independently in four harnesses; a transferable reliability finding the benchmark neither requires nor measures. We offer, as users of the benchmark rather than its authors, five concrete adjustments that would let the community leaderboard score the capability ARC-AGI-3 was built to probe: (i) a cost-conditioned score (RHAE at a fixed budget), not only a peak score; (ii) a standardized token triple (cached input, uncached input, and output), plus dollars at stated prices and wall-clock; (iii) an optional first-attempt or bounded-learning track that measures efficient play directly rather than world-model construction; (iv) verification (scorecard replay plus published traces) as a submission requirement, since a self-reported score cannot separate a derived answer from a looked-up one; and (v) moving the headline to the held-out sets, which several teams (ourselves included) note the public set no longer stands in for. The full write-up, with sources, is in the repository.

6.3 sp80 as an open problem

For most of this project, sp80 was the open problem: below the cap on every board, for us and for Retrodict alike. Our longest text-mode run reached level 5 of 6 with every level at the cap, then spent days on level 6; and its endgame is the scientifically interesting part. The agent stopped probing and started *proving*: under a physics that backtested green on 4,655 of 4,670 recorded transitions, it exhaustively searched on the order of 4.1×10^8 candidate configurations and established that no solution existed under the rules it knew, while isolating the single unexplained observable (a persistent animation-length residual) that had to hide the missing mechanic. The proof was correct; the rules were wrong; and the residual it flagged was, in hindsight, the exact signature of an observation channel that deletes evidence; the hypothesis §4.4 tests directly. We still propose "time-to-missing-mechanic"; the interval between an agent's first proof-grade exhaustion of its rule set and its first observation isolating the missing mechanic; as an honest frontier metric on games like this; on sp80, that interval closed the first session the agent could see.

6.4 Limits

Model identity is pinned by the session transcripts (§4.1). Beyond that: one benchmark; public set only; $n = 1$ per cell; repeated development against the same 25 games; board-composite variance of roughly ±1–2 points in our data; much larger per-game variance (one same-configuration GPT rerun moved from 47.6 to 100.0); and quota-shaped runs resumed across provider windows under a disclosed policy. The deepest caveat is inherited from §5.1: whether the harness *causes* the scores, versus the models alone, is unmeasured until a boundary-correct ablation is run. The public set was both development material and evaluation surface, so these results establish neither held-out generalization nor private-set performance.

7 Related Work

Early ∼99% reports. The first ∼99%-class numbers on this benchmark were self-reported from write-ups without released code, using a per-game best-of-2 fallback with score feedback; the selection rule Kamradt's critique targeted. Our initial loop, run *with* that fallback for comparison only, matched those numbers within noise (98.30 / 95.51); we then abandoned the rule as score-conditioned selection. The §3 ladder reports the same initial loop without fallback selection (97.78). The executable-world-model idea itself is established in published, citable work; Tycho [8] and Rodionov's baseline1 line [13]; of which this project is an independent implementation.

Tycho (NIMI-research; arXiv:2607.28287) [8]; 100.00, the first published perfect score; paper-artifact release with frozen configs and CI, official scorecard per row, traces published as SHA-256 hashes rather than inspectable data; no cost accounting. We adopted (and credit in `NOTICE`) its partial UNKNOWN-cell predictions, vacuous-coverage rejection, threaded hidden-state planning, consecutive-RESET guard, and 5×-baseline level cutoff. Our discipline regression subsequently falsified treating the last item as an *official-rule* limit (§3.1); adopted mechanisms still warrant independent testing.

Retrodict (Ryan Brown) [4]; 99.86 at $654 / 659.9M tokens; the transparency standard this repo aims to meet and exceed: full traces including superseded attempts, per-game cost ledgers, and the best replay disclosure in the field ("a verified re-execution of the recorded runs, not a new attempt"; language we adopt). Its comparison memo publicly critiqued the earliest 99% release; Kepler's release is in large part an answer to that review. We also adopt its sparse expected-cell predictions, escalation-ladder tiers, and prediction-discipline framing (`NOTICE`).

baseline1 / ThinHarness [13]; the minimal-agent lineage Retrodict builds on; used throughout the field as the null scaffold.

Concurrent community submissions. While this paper was in preparation, the pending community-leaderboard entries converged on the same architecture claim (1) describes: GKM freezes model-written programs behind an admission gate and states *"at scoring time no model runs"*; arc-skill's scorecard *"was produced by replaying the recorded runs"*; Strands and arc-skill both run Claude Opus 5, the model our boards resolve to. We read four independent 98-to-100-class systems scoring mechanical replays; none citing each other; as strong evidence that certify-then-replay is the dominant design, which makes stating it explicitly (and auditing it) the more urgent.

VISTA and AVO: the direct-interaction school. Concurrently with this work, VISTA [6] (Han et al., MIT) reported 100.00 (Claude Opus 5, 7,542 actions) and 98.27 (GPT-5.6 Sol) using the opposite architecture to the world-model school: the model observes rendered PNG frames directly, reasons in free-form language with no executable-model requirement, and keeps a lossless visual memory of every frame. NVIDIA's AVO [9] adopted VISTA's direct-interaction principles

(explicitly declining Tycho-style programmatic world models) and reported 100.00 with a supervisor-redirected general coding agent and 6,624 environment actions, lower than VISTA's 7,542. AVO does not publish an ARC implementation, traces, or cost accounting; VISTA publishes a project page and official replay evidence but not a directly comparable cost ledger. Two implications for this paper. First, the replay-scored segmentation of §6.2 spans *both* schools; the convergence is about what is scored, not how games are learned. Second, the schools have never been compared under controlled conditions (AVO: "this is not a controlled ablation"); the observation channel (text grid vs. rendered frames) is confounded with everything else. Our harness records every frame and can toggle the observation channel alone (`-visual`), holding tools, discipline, and budget fixed for one game; §4.4 reports that single-game intervention. It does not establish a benchmark-wide visual advantage.

Prime Agent [12]; a general-purpose, self-improving coding harness adapted to ARC-AGI-3 through its task prompt and broker. Prime Intellect reports three Opus 5 runs at 94.99–95.50, with a 95.24 median scorecard and estimated costs of $944–1,288. This is stronger run-to-run evidence than Kepler's $n = 1$ release cells. Kepler reports a higher single-board score and a stricter integrity record; Prime Agent makes the broader transfer claim across coding and long-context tasks.

arc-code (Berman) [3]; the minimal-scaffold datapoint: stock Claude Code with an 11-line prompt reaches 96.2 pass@1 / 99.3 pass@2 (∼$540), while GPT-5.6 Sol under the same scaffold scores **73.7**. This cuts against attributing Kepler's Claude score entirely to harness design and reinforces the need for a valid control experiment.

8 Reproducibility Statement

The code, official scorecards, release manifest, and verification programs ship in the repository. Corpus-wide verification runs offline once the companion trace dataset is present:

```
.venv/bin/python scripts/audit_integrity.py --traces-dir traces
.venv/bin/python scripts/verify_scores.py --traces-dir traces
python3 scripts/verify.py                    # replay local fixture
.venv/bin/python scripts/cost_report.py      # token accounting
python3 scripts/check_no_game_ids.py         # zero-prior gate
```

The prepared companion trace corpus contains winners, failures, regressions, and superseded runs with their reasons, as append-only `events.jsonl` ledgers plus each agent's `world_model.py` and `notes.md`. Frozen-harness hashes were registered in `RESULTS.md` before their boards' results existed. Headline scorecards are c9f087f3 (GPT, exact) and 91aa2f10 (Opus, exact); historical cards remain indexed in `RESULTS.md`. Bit-for-bit reproduction of live runs is not expected because of sampling and provider drift. Score reproduction from a ledger is mechanical. At the time of this draft, the dataset URL is not public; scripts therefore report a vacuous-data state on fresh clones, and independent full-corpus verification remains incomplete.

Acknowledgments

Ryan Brown (Retrodict) and the Tycho authors (NIMI-research), whose released mechanisms this harness adopts with attribution (`NOTICE`) and whose release engineering set the bar this project measures itself against; the ARC Prize Foundation for ARC-AGI-3, the `arc-agi` toolkit, and the official RHAE implementation.

Appendices. Full per-game tables for all boards, the guard-deadlock and dead-code forensic timelines, the sp80 search inventory, the lf52 replay transcripts, and the audit flag taxonomy are provided in the repository as tables and write-ups. The raw append-only ledgers they were derived from belong to the companion trace corpus, which is prepared but not yet published (§8); until it is, the appendices can be read but not independently recomputed from source data.

References

[1] Anthropic. Claude opus 5: Pricing. Product documentation, 2026. URL https://www.anthropic.com/claude/opus.

[2] ARC Prize Foundation. ARC-AGI-3: Technical report. arXiv:2603.24621, 2026.

[3] Jeremy Berman. arc-code: a minimal 11-line scaffold for ARC-AGI-3. GitHub repository, 2026.

[4] Ryan Brown. Retrodict: an ARC-AGI-3 harness with full trace disclosure. GitHub repository, 2026. URL https://github.com/ryanbbrown/Retrodict.

[5] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[6] Qiushi Han, Keya Hu, Linlu Qiu, Cathy Wu, and Kaiming He. VISTA: A visual harness for reasoning in an interactive world. Project page, 2026. URL https://vista-research.github.io/.

[7] Greg Kamradt. Commentary on early arc-agi-3 harness claims (x post). Post on X (Twitter), 2026. URL https://x.com/GregKamradt/status/2077949388673151332.

[8] NIMI-research. Tycho: A certified-replay harness for ARC-AGI-3. arXiv:2607.28287, 2026.

[9] NVIDIA AVO team. NVIDIA AVO reaches 100% on ARC-AGI-3. NVIDIA Technical Blog, 2026. URL https://developer.nvidia.com/blog/nvidia-avo-reaches-100-on-arc-agi-3-demonstrating-a-frontier-level-general-purpose-architecture-for-long-horizon-autonomous-agents/.

[10] OpenAI. How enabling two settings tripled our scores on the arc-agi-3 benchmark. Blog post, 2026. URL https://openai.com/index/how-two-settings-tripled-our-arc-agi-3-scores/.

[11] OpenAI. GPT-5.6 Sol: Model and pricing. Developer documentation, 2026. URL https://developers.openai.com/api/docs/models/gpt-5.6-sol.

[12] Prime Intellect. Prime agent: A self-improving RLM agent. Project release and technical blog, 2026. URL https://www.primeintellect.ai/blog/prime-agent.

[13] ThinHarness contributors. baseline1 / ThinHarness: minimal-agent scaffolds for ARC-AGI-3. GitHub repository, 2026. URL https://github.com/astroseger/arc-3-agents-baseline1.